Footnote: LA Hacks 2026 Build Plan

Oct 17, 2026 · @Sohail Haresh Gidwani

TL;DR

We are building **Footnote**: an Even G2 app that decodes jargon for new hires in meetings, plus an agent that turns their confusion into follow-up messages. Team of 2 (Sohail, Kartik), hacking runs Sat 12:00 PM to Sun 12:00 PM.

Pitch: You hear "the BRT cutover is blocked on SOC2" in your first week. One line appears in your glasses: what it means and who owns it. One tap queues a question for that person. When the meeting ends, the agent sends each owner one short message. Terms you learn fade out, so the app gets quieter as you ramp up.

Prize targets, in priority order:

Tier	Prize	What earns it
Core	Even Realities challenge	Working G2 loop, Beta build, judge can try it at our table
Core	Workday challenge	Helps a new hire ramp, find trusted info, act on it
Core	Beyond the Digital track	Wearable AI
Cheap add-on	StepFun	step-5-preview explains unknown terms
Cheap add-on	MLH Best Domain (GoDaddy Registry)	Register a domain in hour 1
Should	MLH Best Use of ElevenLabs	Scribe speech-to-text
Stretch	Fetch.ai	One agent on Agentverse, usable from ASI:One
Stretch	Paper	G2 cards and digest page designed in Paper

Go/no-go checkpoints:

- **12:30 PM Sat:** G2 approval email. No device means switching to the TrialScout fallback (section 7).
- **6:00 PM Sat:** Hello-world text renders on the real G2 from our app, and the backend is live on a public HTTPS URL.
- **8:00 PM Sat:** Full core loop on glasses (tap, listen, card, tap to queue). Throwaway Beta build uploaded while the Even table is still open.
- **9:00 AM Sun:** Feature freeze. Only bug fixes, demo video, Devpost after this.

Product

Footnote does four things in order: detect a term, show one line, let you act with one press, then follow through after the meeting. Every feature below serves that loop; anything that doesn't is cut.

The loop

1. **Listen.** Single press on the temple starts listening mode. The G2 four-mic array streams audio to our backend.
2. **Detect.** The backend transcribes rolling ~5 s chunks and finds terms from the company glossary (fast path) or unknown acronyms and capitalized names (slow path, StepFun).
3. **Show.** One card on the glasses: the term, a 1-line meaning, and the owner. Example: `BLUEBIRD` / `v3 navigation stack rewrite` / `Owner: Priya R. (Autonomy)`.
4. **Act.** Single press while a card is showing queues a question for the owner. The card flips to `Queued for Priya ✓`. Swipe down means "I know this": the term is muted for good.
5. **Follow through.** Long press (or the phone screen) ends the meeting. The backend drafts one batched message per owner, and on approval sends it.
6. **Fade out.** Each term has a seen counter. After 3 sightings, or one swipe-down, it stops appearing. The app literally gets quieter as you learn.

Gesture map

Gesture (SDK event)	Idle	Card showing
Single press (`CLICK_EVENT`)	Start / stop listening	Queue question for owner

Gesture (SDK event)	Idle	Card showing
Swipe down (`SCROLL_BOTTOM_EVENT`)	Nothing	"I know this": mute term
Swipe up (`SCROLL_TOP_EVENT`)	Show last card again	Show the next card in the queue
Long press (`LONG_PRESS_EVENT` / `_RELEASE`)	End meeting, show summary	End meeting, show summary

Double press is reserved for exit: on the root page it must call `shutDownPageContainer(1)`, which shows the system exit dialog. Reviewers reject apps that exit silently or use a custom exit UI. Long press is not guaranteed either (the OS can claim tap-then-hold for its contextual menu), so "end meeting" also needs a button on the phone screen.

Features by priority

Priority	Feature	Owner
Must	Listening mode with G2 mic, status line on glasses (`● Listening`)	Sohail
Must	`/listen` endpoint: STT, glossary match, returns cards	Kartik
Must	Card display + press to queue + swipe to mute	Sohail
Must	Synthetic company pack: ~40 glossary terms, ~12 people, ~8 projects	Kartik
Must	StepFun explains unknown terms using the company pack as context	Kartik
Must	End-meeting: batched messages drafted, approved, sent to our own inbox	Kartik
Must	Phone screen (the WebView page): meeting controls, queued questions, approve-and-send	Sohail
Must	Seen-count fade-out	Kartik
Should	ElevenLabs realtime STT (WebSocket) to cut card latency below ~2 s	Kartik

Priority	Feature	Owner
Should	Digest web page: terms learned today, people to meet, messages sent	Sohail
Stretch	Fetch.ai Footnote Agent: briefing + approve-and-send from ASI:One	Kartik
Stretch	Card system + digest redesigned in Paper	Sohail

Not building

- Real Slack, Teams or Gmail integration with OAuth. We send email from one account to our own inbox.
- Calendar booking. The message proposes a coffee chat in text instead.
- Speaker identification, meeting summaries, full transcripts. Even says note-takers are crowded; we are not one.
- Accounts, multi-user, multi-company. One hardcoded user and one company pack.

Demo script (live at our table, ~90 s)

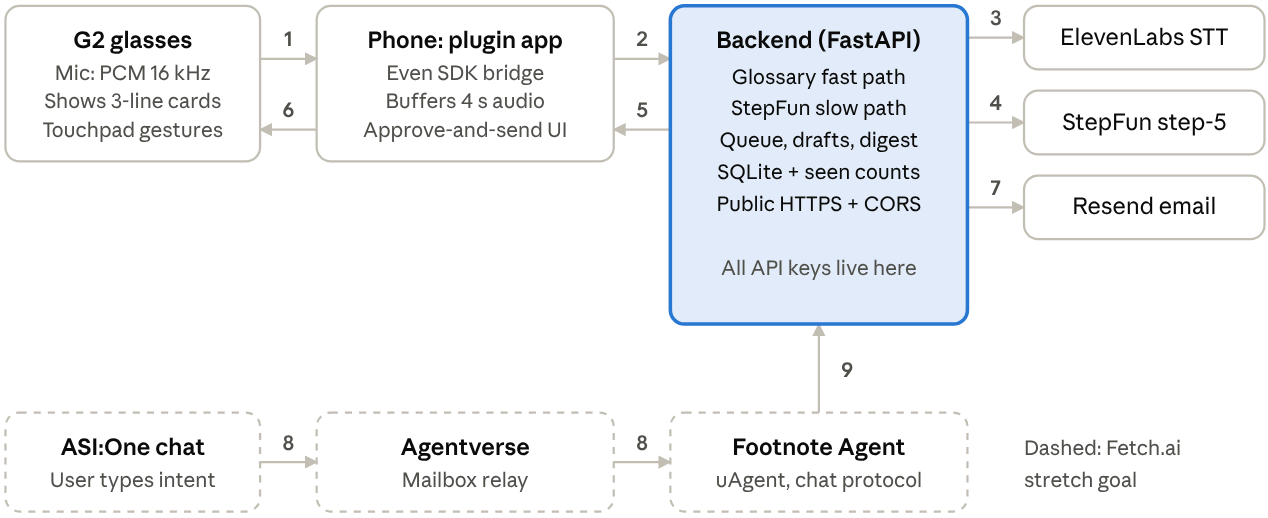
1. Hand the judge the G2. "You started at Halcyon Robotics on Monday. You're in your first sync."
2. Judge presses once: `● Listening` appears.
3. Kartik reads: "Bluebird is slipping because FCS is still on the old golden path, so we might punt to the Q4 bake-off."
4. Cards appear for BLUEBIRD, then FCS. Judge presses once on Bluebird: `Queued for Priya ✓`.
5. Judge swipes down on the FCS card ("I know this"). Kartik says "FCS" again: no card. That's the fade-out.
6. Long press (or phone button): `Meeting ended · 2 questions queued`. On the phone, approve, and the email lands in our inbox on the laptop screen.
7. Close: "Footnote gets quieter every day. By week three, you never see it."

The 2 to 3 minute Devpost video follows the same script, with through-the-lens or simulator footage of the glasses.

Architecture

The glasses and phone are thin clients: they move audio up and text down. Every API key, every model call and all state live in one FastAPI backend, which is what Even's packaging rules require anyway (anything in the `.ehpk` can be extracted).

One backend holds every key; glasses and phone only move audio and text



Footnote system · 9 numbered data flows

Reading the numbers. In the meeting: (1) the G2 mics stream PCM and gestures to the phone; (2) the phone posts each ~4 s chunk to `/listen`; (3) the backend transcribes it; (4) unknown terms go to StepFun; (5) cards come back; (6) the phone pushes the card to the glasses with `textContainerUpgrade`. After the meeting: (7) approved drafts go out by email through Resend. Stretch: (8) a user chats in ASI:One, relayed through the Agentverse mailbox to our agent, which (9) calls the same backend REST endpoints.

In-meeting flow and latency budget

The fast path never waits for the slow path. Glossary cards return in the same `/listen` response; StepFun runs in the background and its cards ride on the **next** `/listen` response.

Step	Where	Target
Word spoken to chunk sent	Phone (4 s buffer)	2 s average wait

Step	Where	Target
Upload chunk (~128 KB)	Phone → backend	< 300 ms
Speech-to-text	Backend → ElevenLabs	< 1,000 ms
Glossary match	Backend	< 20 ms
Card to glasses	Phone → G2 over BLE	< 300 ms
Total, glossary term		**≈ 3.5 s**
StepFun explanation (background)	Backend → StepFun	capped at 2.5 s, shown on next chunk

If the total is over 4 s in the backend timing logs, cut the chunk to 2.5 s first; switching to Scribe Realtime is the bigger fix.

Glasses layout (576 × 288, one page)

Container	Position (x, y, w, h)	Brightness	Example content
`status` (id 1)	0, 0, 576, 40	2 (dim)	`● Listening · 2 queued`
`term` (id 2, captures input)	0, 72, 576, 56	4 (bright)	`BLUEBIRD`
`detail` (id 3)	0, 132, 576, 120	3	`v3 navigation stack rewrite` / `Owner: Priya R. (Autonomy)` / `press: ask · swipe down: known`

All updates use `textContainerUpgrade` with matching `containerID` and `containerName`, or they silently do nothing. Characters outside the firmware font are dropped, so check `●`, `✓` and `▸` against the glyph table in the design guidelines before relying on them. Empty state: `term` and `detail` set to a single space, status stays.

Data model (SQLite)

Table	Columns	Notes
`people`	id, name, team, email	Seeded from company pack; emails are display-only, all mail goes to DEMO_INBOX
`terms`	id, term, aliases, meaning, owner_id, source	`source` = glossary or llm (llm terms are cached after first explanation)
`sessions`	id, started_at, ended_at	One per meeting
`seen`	term_id, count, muted	Drives the fade-out: hide when count ⩾ 3 or muted
`queue`	id, session_id, term_id, owner_id, created_at	One row per press
`drafts`	id, session_id, owner_id, subject, body, status	status = draft or sent

Railway and Render disks reset on redeploy; that's fine, because the backend reseeds from `/data/company_pack.json` on startup.

Repo layout

```
footnote/
  CLAUDE.md            shared rules + API contract (section 5)
  README.md            includes "Run on G2" section
  glasses/             Sohail: Vite + TS + Even SDK, app.json
  web/                 Sohail: digest page (stretch)
  backend/             Kartik: FastAPI, Dockerfile, .env (never committed)
  agent/               Kartik: uAgent + Agentverse README (stretch)
  data/                Kartik: company_pack.json
```

The full API contract lives in `CLAUDE.md` in section 5, so both coding agents read the same copy.

Work split and timeline

Sohail owns everything that runs on the phone and glasses; Kartik owns everything that runs on the server. The only thing you share is the API contract in section 3, frozen at 12:30 PM. Swap roles if Kartik is stronger in TypeScript.

	Sohail (glasses + phone)	Kartik (backend + AI + agent)
Folder	`/glasses`, `/web`	`/backend`, `/agent`, `/data`
Language	TypeScript (Vite, Even SDK)	Python 3.11 (FastAPI, uAgents)
Agent skills	`everything-evenhub` plugin, Paper MCP	ElevenLabs skill
Owns sponsors	Even Realities, Paper	StepFun, ElevenLabs, Fetch.ai, domain, email (Resend)
Demo role	Explains the G2 and hands it over	Reads the jargon line, shows the inbox

Hour by hour (Sat 12:00 PM to Sun 12:00 PM)

Time	Sohail	Kartik	Gate
12:00 to 12:30	Create repo, paste `CLAUDE.md`, install Even CLI + simulator + plugin; pick up G2 if approved	Get keys (ElevenLabs, StepFun, Resend, ASI:One, Agentverse), register domain	**API contract frozen**
12:30 to 3:00	Scaffold from `asr` template; card layout and gestures in the simulator against a mock `/listen`	FastAPI skeleton deployed on public HTTPS with CORS; company pack JSON; glossary matcher; `/listen` with text input	Mock card shows in simulator
3:00 to 4:00	Even workshop (SDK, testing on G2)	ElevenLabs batch STT on raw PCM	
4:00 to 6:00	Pair G2, QR sideload, stream G2 mic PCM to `/listen`	StepFun slow path; seen counts; queue endpoint	**6 PM: real text on real G2, backend public**

Time	Sohail	Kartik	Gate
6:00 to 8:00	Real cards on glasses, press to queue, swipe to mute; throwaway Beta upload	End meeting: draft + send batched emails via Resend	**8 PM: full core loop on G2, Beta pipeline proven**
8:00 to 11:00	Phone screen: listening state, queued questions, approve and send	Latency pass using backend timing logs; realtime STT if batch is over ~3 s	Midnight check-in
11:00 to 3:00	Digest page; Paper redesign of cards (stretch)	Fetch.ai Footnote Agent + ASI:One flow (stretch)	
3:00 to 7:00	Sleep 3:00 to 6:00	Sleep 4:00 to 7:00	One person always awake
7:00 to 9:00	10 full demo runs on the real G2; fix only what breaks	Same, from the backend side	**9 AM: feature freeze**
9:00 to 10:30	Record demo video (through-the-lens + simulator)	README (G2 install section, keys), Devpost text	
10:30 to 11:30	Final `.ehpk` to Beta, install on phone, retest	ASI:One shared chat URL, Agentverse profile URL	
11:30 to 12:00	Submit Devpost + EvenHub Beta together	Submit	**Noon: hacking ends**

If a gate is missed: at 6 PM, drop realtime STT and Fetch entirely. At 8 PM, drop the digest page and Paper; demo the follow-up email from the phone screen only. Never trade sleep for a stretch feature.

Starter prompts

Paste the shared `CLAUDE.md` into the repo root first, then run your prompts in order, one per agent session. Each prompt ends with a check you can see; don't start the next prompt until it passes.

Shared `CLAUDE.md` (repo root)

```
# Footnote

Even G2 smart-glasses app for new hires. During a meeting it listens, detects
unfamiliar company terms, shows one line on the glasses (meaning + owner),
lets the user queue a question with one press, and after the meeting sends
each owner one batched message. Terms fade out after 3 sightings or a mute.
Demo company: "Halcyon Robotics" (synthetic data in /data/company_pack.json).

## Repo layout (stay in your folder)
/glasses  TypeScript, Vite, @evenrealities/even_hub_sdk. Owner: Sohail.
/web      Digest page (React + Vite). Owner: Sohail.
/backend  Python 3.11 FastAPI. Owner: Kartik.
/agent    Fetch.ai uAgent (chat protocol, mailbox). Owner: Kartik.
/data     company_pack.json. Owner: Kartik.
Never edit another owner's folder. If you need a contract change, stop and
say so.

## API contract (frozen; backend base URL in env BACKEND_URL, HTTPS)
POST /session/start                       -> {session_id}
POST /listen?session_id=ID                body: raw PCM bytes (16 kHz, s16le,
mono, ~4 s)
                                          -> {transcript, cards: Card[]}
POST /listen_text {session_id, text}      -> {transcript, cards: Card[]}
(testing + fallback)
POST /cards/{card_id}/queue {session_id} -> {queued: true, owner_name}
POST /terms/{term_id}/mute                -> {muted: true}
POST /session/{session_id}/end            -> {drafts: Draft[]}
POST /drafts/{draft_id}/send              -> {sent: true}
GET  /digest                              -> {terms_learned[],
people_to_meet[], sent[]}
GET  /health                              -> {ok: true}

Card  = {card_id, term_id, term, line1, owner_name, owner_team, source:
"glossary"|"llm", confidence}
        term <= 24 chars, line1 <= 44 chars, owner_name + owner_team <= 40
chars
Draft = {draft_id, to_name, to_email, subject, body, terms[]}

A term is never returned as a card if muted or seen >= 3 times.
The backend never returns more than 2 cards per /listen call.
```

```
## Hard rules
- No API keys in /glasses or /web. All third-party calls go through /backend.
- Backend sends CORS headers for every origin and answers OPTIONS preflights.
- Glasses UI: plain text only, no bold/size control; brightness via textColor
0-4.
- Root-page double press calls shutDownPageContainer(1). Never exit silently.
- Target: card on glasses within 3 s of the word being spoken.
- Email: Resend API only; every draft goes to env DEMO_INBOX. Company-pack
emails are display-only.
- Logging: plain Python logging with per-step timings (stt_ms, match_ms,
llm_ms). No Sentry.
- Prefer the simplest thing that demos reliably. No new dependencies without
reason.
```

Sohail: one-time setup (12:00 PM)

1. `npm install -g @evenrealities/evenhub-cli` and install the simulator per the Even docs.
2. In Claude Code: `/plugin marketplace add even-realities/everything-evenhub`, then `/plugin install everything-evenhub@everything-evenhub`. Codex works too ($ prefix instead of /).
3. Stretch, for Paper: install Paper Desktop, open a file, then `/plugin marketplace add paper-design/agent-plugins` and `/plugin install paper-desktop@paper`.

Sohail: prompts

S1. Scaffold (12:30 PM)

> Read CLAUDE.md. Use /template --asr to scaffold the app into /glasses as "footnote". Set app.json: package_id com.footnote.app, name "Footnote", permissions g2-microphone and network (whitelist only BACKEND_URL's origin). Build a root page with three text containers: status (top, dim, textColor 2), term (middle, textColor 4, isEventCapture 1), detail (below, textColor 3). Use textContainerUpgrade for all updates. Mock the backend with a local function that returns two hardcoded Cards from the contract. Run it in the simulator. Check: I can see a card render in the simulator.

S2. Gestures (by 3 PM)

> Implement the gesture map: single press idle = start/stop listening; single press with a card = POST /cards/{id}/queue and show "Queued for {owner} ✓" for 2 s; swipe down with a card = POST /terms/{id}/mute and clear; swipe up = show the next queued card; long press = POST /session/{id}/end; double press on root = shutDownPageContainer(1). Cards auto-clear after 8 s with no action. Use /handle-input. Check: every gesture works in the simulator, including long press.

S3. Real mic (4 PM, on the G2)

> Use /device-features. On start listening, call audioControl(true, AudioInputSource.Glasses) after the startup page exists. Buffer audioEvent.audioPcm (16 kHz s16le mono) into ~4 s chunks (128,000 bytes) and POST each to /listen?session_id= as application/octet-stream. Show each returned card; queue extra cards. If a request takes over 6 s, drop it. Show "● Listening" and a small error state when the backend is unreachable. Check: on the real G2, saying "Bluebird" makes the Bluebird card appear.

S4. Phone screen (6 PM)

> The plugin's HTML page is the phone UI. Build it: big Start/End meeting buttons, list of queued questions, and after End, the drafts with an Approve and send button each (POST /drafts/{id}/send). Keep it one file, no framework. Check: I can end a meeting and send a draft from the phone.

S5. Package (7:30 PM, repeat at 10:30 AM)

> Use /build-and-deploy. Run npm run build, then evenhub pack app.json dist --sdk-ver <installed SDK version> -o footnote.ehpk. Tell me exactly what to upload in the developer portal for a Beta build. Check: the Beta build installs from the phone app and runs.

S6. Digest page (stretch)

> Build /web: one page that calls GET /digest and shows terms learned today, people to meet, and messages sent. Calm, minimal, green-on-dark to echo the G2. Check: it renders with live data from the deployed backend.

Kartik: one-time setup (12:00 PM)

1. Keys into `/backend/.env` (never committed): `ELEVENLABS_API_KEY`, `STEPFUN_API_KEY`, `RESEND_API_KEY`, `DEMO_INBOX`, `ASI_ONE_API_KEY`, `AGENTVERSE_API_KEY`.
2. Email: sign up for Resend with **sohailgidwani16@gmail.com**. Without a verified domain, Resend only delivers to the signup address, so set `DEMO_INBOX=sohailgidwani16@gmail.com` and send from `onboarding@resend.dev`. Keep the address in `.env` only, not in the public repo.
3. Optional agent skill: `npx skills add elevenlabs/skills --skill speech-to-text`.

Kartik: prompts

K1. Skeleton + deploy (12:30 PM)

> Read CLAUDE.md. Create /backend: FastAPI with every endpoint in the contract returning correct-shaped stub data, CORS allow all origins/methods/headers, SQLite via sqlite3 (tables: people, terms, sessions, seen, queue, drafts), seeded from /data/company_pack.json on startup. Add a Dockerfile and deploy it to Railway (or Render) so it has a public HTTPS URL. Check: curl https://<url>/health returns ok, and an OPTIONS preflight returns 204 with CORS headers.

K2. Company pack (12:45 PM)

> Write /data/company_pack.json for a fictional company, Halcyon Robotics: 12 people (name, team, email = firstname@halcyon.example, display only, since all mail really goes to DEMO_INBOX), 8 projects, 40 glossary terms (term, aliases, one-line meaning <= 44 chars, owner person id). Must include: Bluebird (v3 navigation stack rewrite, owner Priya R., Autonomy), FCS (Fleet Control Service), golden path, Q4 bake-off, SEV2, Kestrel, DRI, the 2pm platform sync. Make terms realistic and mutually consistent. Check: JSON validates and every owner id exists.

K3. Fast path (2 PM)

> Implement /listen_text: normalize the transcript, match glossary terms and aliases with rapidfuzz (token_set_ratio >= 90) plus exact acronym match, skip muted or seen >= 3, increment seen, return at most 2 cards. Then /listen: send the raw PCM to ElevenLabs speech-to-text (batch; raw 16 kHz s16le PCM, or wrap it in a WAV header if the API needs one), then run the same matcher. Log timings for STT and match separately. Check: posting a recorded 4 s PCM clip of someone saying "Bluebird" returns the Bluebird card.

K4. Slow path with StepFun (4 PM)

> For capitalized words and 2 to 5 letter acronyms in the transcript that are NOT in the glossary, call StepFun step-5-preview through the OpenAI Python SDK (base_url https://api.stepfun.ai/v1, model step-5-preview, low reasoning effort, JSON schema output). Put the whole company pack in the system prompt. Ask: is this a company-specific term, its meaning in <= 44 chars, most likely owner from the people list, and confidence 0 to 1. Only return cards with confidence >= 0.6. Never block the response on it: run it in the background, cap it at 2.5 s, store its cards, and return them with the next /listen response for that session. Check: "Did Marcus update the HAL-9 runbook?" returns a plausible llm card.

K5. Queue, end meeting, send (6 PM)

> Implement /cards/{id}/queue, /terms/{id}/mute, /session/{id}/end and /drafts/{id}/send. End meeting groups queued terms by owner and asks StepFun to write one short, friendly message per owner from a new hire (2 to 4 sentences, asks for a 2-line summary or 15 minutes this week). Send with the Resend API (POST https://api.resend.com/emails, from onboarding@resend.dev) to DEMO_INBOX, putting the owner's name in the subject, e.g. "To Priya R.: quick question about Bluebird". Check: ending a session with 2 queued terms for Priya produces one draft, and sending it lands in our inbox.

K7. Fetch.ai agent (stretch, after 11 PM)

> Create /agent: a uAgent with mailbox=True and publish_agent_details=True using the chat protocol from uagents_core.contrib.protocols.chat (ack every ChatMessage, reply with TextContent). Use StepFun to route each user message to one of: brief me for a meeting (GET a list of likely terms + owners from the backend), what does X mean here, show my queued questions, send my questions (show drafts, send only after the user replies yes). Write a README with the innovationlab and hackathon badges, capability keywords and example queries. Check: chatting with the agent from ASI:One sends a real email after I say yes.

Sponsor setup guides

Each guide says what the sponsor needs to see, the exact setup, and the gotchas that cost teams hours. Requirements are taken from each sponsor's own pages, linked at the end.

Even Realities G2 (core)

Judges want: an app built for G2 that the judge can try at our table, plus answers to "who uses this, when, and why G2". Rubric: Made for G2, Originality, User Experience, Technical Execution. Submit **both** Devpost and an EvenHub Beta build by noon.

Facts that shape our design:

- Canvas is 576 × 288 per eye, 4-bit greyscale green. Text containers only: no font size, no bold, no alignment, no background. Brightness via `textColor` 0 to 4.
- Max 8 non-image containers per page; exactly one has `isEventCapture: 1`.
- A full-screen text container holds roughly 400 to 500 characters, so a card is 3 short lines at most.
- Use `textContainerUpgrade` for updates (flicker-free); `rebuildPageContainer` flickers and wipes state.
- Mic: `audioControl(true, AudioInputSource.Glasses)` streams PCM 16 kHz, s16le, mono on `event.audioEvent.audioPcm`. Glasses mic needs `createStartUpPageContainer` to have run first. Phone mic is the fallback (`AudioInputSource.Phone`).
- Gestures: press, double press, swipe up/down, long press (SDK 0.0.14+). Long press can be claimed by the OS contextual menu, so never make it the only path.
- The glasses have no speaker and no camera.

Networking (two gates, both must pass): the backend origin must be in `app.json` → `permissions` → `network.whitelist` (full origin like `https://footnote-api.up.railway.app`, no wildcards), **and** the backend must send CORS headers and answer `OPTIONS` with 204. Changing the whitelist means repacking and reuploading.

Manifest rules: `package_id` lowercase reverse-domain with no hyphens (`com.footnote.app`), `edition` `202601`, `name` 20 characters or fewer and must not contain "Even", `version` x.y.z, `min_sdk_version` required. Declare only `g2-microphone` and `network`.

Commands:

- Dev loop: `npm run dev`, then `evenhub qr --url "http://<laptop LAN IP>:5173"` and scan with the Even Realities app. If venue Wi-Fi blocks this, put the laptop and phone on a phone hotspot.
- Ship: `npm run build`, then `evenhub pack app.json dist --sdk-ver <your SDK version> -o footnote.ehpk`. Upload under Builds in the developer portal, push to a Beta group with our phone account, install from Me → Beta tester.
- Before submitting, test: recovery after a 5-minute phone lock, denied mic permission, and double press exit on the root page.

Never put API keys in the `.ehpk`; anyone can unpack it. Everything goes through our backend.

Workday (core, no SDK)

Judges want: an AI agent or prototype that helps people succeed after joining a company: ramping into a role, finding trusted information, completing everyday tasks, building skills. **How we hit it:** the pitch and Devpost should name each of those words. Trusted information = answers come from the company pack with an owner attached, not from the open web. Ramping = the fade-out. Everyday task = the follow-up message.

StepFun (cheap add-on)

Judges want: the app must call `step-5-preview` when it runs; explain what each model does; they judge task completion, reliability, and whether users can understand and control the workflow.

- OpenAI-compatible. Python: `OpenAI(base_url="https://api.stepfun.ai/v1", api_key=STEPFUN_API_KEY)`, `model="step-5-preview"`. If the key or region gives trouble, OpenRouter serves it as `stepfun/step-5-preview`.

- Supports JSON Schema output, tool calling, and low/medium/high reasoning effort. Use **low** effort for term explanation (latency) and **medium** for drafting messages.
- 1M token context with prompt caching: put the whole company pack in the system prompt instead of building RAG.
- **User control** (part of their rubric): the user approves every message before it sends, and can mute any term. Say this out loud in the demo.

ElevenLabs (MLH prize)

- Start with **batch** speech-to-text on each ~4 s chunk; it's the simplest thing that works.
- If latency is too high, switch to **Scribe v2 Realtime** over WebSocket from the backend (server-side streaming uses the normal API key). Set `audio_format` to `pcm_16000`, which matches the G2 mic output exactly, so no resampling. Partial transcripts arrive while speaking, which lets the matcher fire before the sentence ends.
- Agent skill: `npx skills add elevenlabs/skills --skill speech-to-text`.

Fetch.ai (stretch)

Judges want: at least one agent registered on Agentverse, implementing the Agent Chat Protocol, discoverable and usable through ASI:One. Submit a **public ASI:One shared chat URL** showing the full workflow and the **Agentverse profile URL**. They also weigh whether the agent could keep operating after the hackathon.

- `pip install uagents`. Chat protocol imports come from `uagents_core.contrib.protocols.chat` (`ChatMessage`, `ChatAcknowledgement`, `TextContent`, `EndSessionContent`, `chat_protocol_spec`).
- Create `Agent(name=..., seed=..., port=8001, mailbox=True, publish_agent_details=True)`, build `Protocol(spec=chat_protocol_spec)`, acknowledge every `ChatMessage`, reply with `TextContent`, then `agent.include(protocol, publish_manifest=True)`.
- **Mailbox mode** means the agent runs on a laptop with no public endpoint: start it, open the Agent Inspector link it prints, click Connect → Mailbox. Then Agent Profile → Chat with Agent opens ASI:One.
- The agent must be **running** for ASI:One to reach it. Keep it running on a spare laptop or the backend host during judging.
- README on Agentverse must include both badges, `![tag:innovationlab](https://img.shields.io/badge/innovationlab-3D8BD3)` and `![tag:hackathon](https://img.shields.io/badge/hackathon-5F43F1)`, plus clear capability keywords

and example queries; discovery depends on them.
- Agentverse also offers an MCP server (`https://mcp.agentverse.ai/sse`) if you want your coding agent to manage the listing.

Paper (stretch)

Judges want: best-designed interface, judged on design fundamentals, ease of use and personal craft.

- Paper Desktop must be running with a file open. Connect Claude Code with `claude mcp add paper --transport http http://127.0.0.1:29979/mcp --scope user` , or install the official plugin (`paper-design/agent-plugins` , `paper-desktop@paper`). The MCP can both read and write the canvas.
- Design two frames: the G2 card system at exactly 576 × 288 (term, detail, queued state, listening state, empty state) and the digest page. Then have the agent implement from the Paper frames.
- **Ask at the Paper table first** whether the design must be made in Paper or only presented. Keep before/after screenshots for Devpost; craft is easiest to show as iteration.

GoDaddy Registry domain (MLH, 10 minutes)

Grab the free domain through the MLH offer at check-in or the MLH table, and confirm which TLDs count. Point it at the digest page. Ideas to check for availability: `footnote.tech` , `getfootnote.xyz` , `quietfootnote.app` .

Sources

Even challenge · Even starter pack · Even display · Even device APIs · Even networking · Even packaging · Even page lifecycle · Even CLI · everything-evenhub skills · uAgents ASI:One example · Agentverse SDK · Fetch hackpack · Paper MCP · ElevenLabs realtime STT · StepFun API

Risks, fallbacks and submission

The biggest risk is not code; it's the plumbing between phone, glasses and backend. Every risk below has a fallback that still produces a working demo.

Risk	Early warning	Fallback
No G2 approved at 12:30	No pickup email	Switch to TrialScout v2 (Fetch + Paper + StepFun), plan in the earlier chat
Phone can't reach laptop dev server	QR loads a blank page	Phone hotspot for both devices; or deploy `/glasses` build to a public URL
Requests blocked on device	Works in simulator, fails on G2	Check whitelist origin, CORS, OPTIONS 204, in that order; repack after whitelist changes
STT too slow (> 3 s per card)	Timing logs show STT dominating	Shorter chunks (2.5 s); switch to Scribe Realtime with partial transcripts
Room too loud for G2 mics	Garbled transcripts	Speaker stands close; switch to `AudioInputSource.Phone` held near the speaker; last resort, `/listen_text` from the phone with typed text
Long press claimed by OS menu	Long press does nothing	End meeting from the phone screen button
StepFun key or region issue	401 or timeouts	OpenRouter `stepfun/step-5-preview`; fast path still works without it
Resend send fails	Auth error	Show the drafted message on the phone; fall back to Gmail SMTP with an app password on the personal Gmail
Beta upload fails at 11 AM	Portal error	We uploaded a throwaway build at 8 PM, so the pipeline is proven; worst case, demo via QR sideload and note it
Fetch agent offline during judging	ASI:One gets no reply	Keep it running on a second laptop on power; record the ASI:One flow in the demo video too

TrialScout fallback in one line

If there's no G2: ASI:One-discoverable agent that finds recruiting trials on ClinicalTrials.gov, asks the fewest follow-up questions (pick each question by how many trials it eliminates), and drafts a pre-screen email to the coordinator, sent to our own

inbox. Same Kartik backend skills, same StepFun wiring.

Submission checklist (Sunday)

- [ ] Final `.ehpk` uploaded as **Beta**, installed from Me → Beta tester, tested on G2 (phone lock 5 min, mic permission denied, double press exit)
- [ ] App name in Even Hub is 20 characters or fewer, no "Even": **Footnote**
- [ ] Devpost: Even Realities challenge selected, Even Hub app name included
- [ ] Devpost: Workday, StepFun, MLH ElevenLabs, MLH domain selected; Fetch and Paper only if built
- [ ] Devpost track: Beyond the Digital
- [ ] Demo video 2 to 3 minutes, showing G2 (through-the-lens or simulator) and the email landing
- [ ] Public GitHub repo; README has a **"Run on G2"** section with accounts and keys a tester needs
- [ ] Devpost has one short paragraph per sponsor: what we used and why
- [ ] StepFun paragraph explains what each model does (StepFun, ElevenLabs)
- [ ] If Fetch: public ASI:One shared chat URL + Agentverse profile URL; agent README has both badges
- [ ] If Paper: before/after frames from Paper in the Devpost gallery
- [ ] Both of us at the table after noon, G2 charged, demo script rehearsed
- [ ] Return the G2 to the Even table before leaving